

Phase 2: Build a Basic Working Agent

1. Objective

The objective of this phase is to build a basic Banking Support Agent using Python. The agent accepts user queries, generates predefined responses using simple rules and templates, refuses unsupported or unsafe requests, and logs interactions safely. This implementation serves as a baseline prototype for understanding customer support workflows and identifying the limitations of a rule-based approach.

2. Agent Design

The Banking AI Support & Advisory Agent follows a simple rule-based architecture.

Workflow:

1. User enters a banking-related query.
2. Agent analyses the query using predefined rules.
3. Agent generates a response.
4. Unsafe or unsupported requests are refused or escalated.
5. User interactions are logged.
6. Sensitive information is masked before logging.

Architecture:

User Input → Rule Engine → Response Generation → Safe Logging

3. Python Implementation

```
import logging
import re

# Configure logging
logging.basicConfig(
    filename="banking_agent_logs.txt",
    level=logging.INFO,
    format="%(asctime)s - %(message)s"
)

# -----
# PII Masking
# -----
def mask_pii(text):
    """
    Mask sensitive information before logging.
    """

    # Mask long numbers (account numbers)
    text = re.sub(r"\b\d{8,}\b", "[ACCOUNT_MASKED]", text)

    # Mask email addresses
    text = re.sub(
        r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}",
        "[EMAIL_MASKED]",
        text
    )

    return text

# -----
```

```

# Response Generator
# -----
def generate_response(user_input):
    """
    Generate rule-based banking responses.
    """

    query = user_input.lower()

    # Home Loan Documents
    if "home loan" in query and "document" in query:
        return (
            "Typical home loan documents include "
            "identity proof, address proof, income proof, "
            "bank statements, and property documents."
        )

    # Travel Reward Credit Card
    elif "travel reward" in query or "travel card" in query:
        return (
            "Travel reward credit cards generally offer "
            "air miles, reward points, and airport lounge access."
        )

    # EMI Query
    elif "emi" in query:
        return (
            "EMI support is available, but this baseline version "
            "cannot perform EMI calculations yet."
        )

    # Fraud or Double Charge
    elif "charged twice" in query or "fraud" in query:
        return (
            "This issue requires human review. "
            "Please contact customer support immediately. "
            "Escalation Recommended."
        )

    # Money Transfer Request
    elif "transfer" in query:
        return (
            "I cannot perform banking transactions. "
            "Please use official banking channels."
        )

    # Balance Inquiry
    elif "balance" in query:
        return (
            "I cannot access customer-specific account information."
        )

    # Default Response
    else:
        return (
            "I am unable to answer that request. "
            "Please contact customer support."
        )

# -----
# Main Agent Loop
# -----
def main():

    print("=" * 50)

```

```

print("Banking Support Agent")
print("Type 'exit' to quit")
print("=" * 50)

while True:

    user_input = input("\nCustomer: ")

    if user_input.lower() == "exit":
        print("Agent: Session Ended")
        break

    response = generate_response(user_input)

    # Mask sensitive information before logging
    safe_input = mask_pii(user_input)

    logging.info(
        f"Customer={safe_input} | Agent={response}"
    )

    print(f"Agent: {response}")

if __name__ == "__main__":
    main()

```

4. Sample Agent Interactions

The screenshot shows the VS Code interface with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project named 'WORK' with files 'agent_logs.txt', 'banking_agent_...', and 'python_based_a...'. The code editor displays the 'python_based_agent.py' file, which contains logging configuration and a main loop. The terminal shows the execution of the script, displaying a 'Banking Support Agent' prompt and several sample interactions between a customer and an agent.

```

vscode
File Edit Selection View Go Run Terminal Help
python_based_agent.py x
python_based_agent.py > ...
1 import logging
2 import re
3
4 # Configure logging
5 logging.basicConfig(
6     filename="banking_agent_logs.txt",
7     level=logging.INFO,
8     format="%(asctime)s - %(message)s"
9 )
10
11 # =====
12 # XXX Masking
13
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
labuser@vscode:~$ vim python_based_agent.py
labuser@vscode:~$ python python_based_agent.py

Banking Support Agent
Type 'exit' to quit
=====

Customer: What documents are required for a home loan?
Agent: Typical home loan documents include identity proof, address proof, income proof, bank statements, and property documents.

Customer: Which credit card offers travel rewards?
Agent: Travel reward credit cards generally offer air miles, reward points, and airport lounge access.

Customer: What is the EMI for a ₹5 lakh loan?
Agent: EMI support is available, but this baseline version cannot perform EMI calculations yet.

Customer: My account was charged twice.
Agent: This issue requires human review. Please contact customer support immediately. Escalation Recommended.

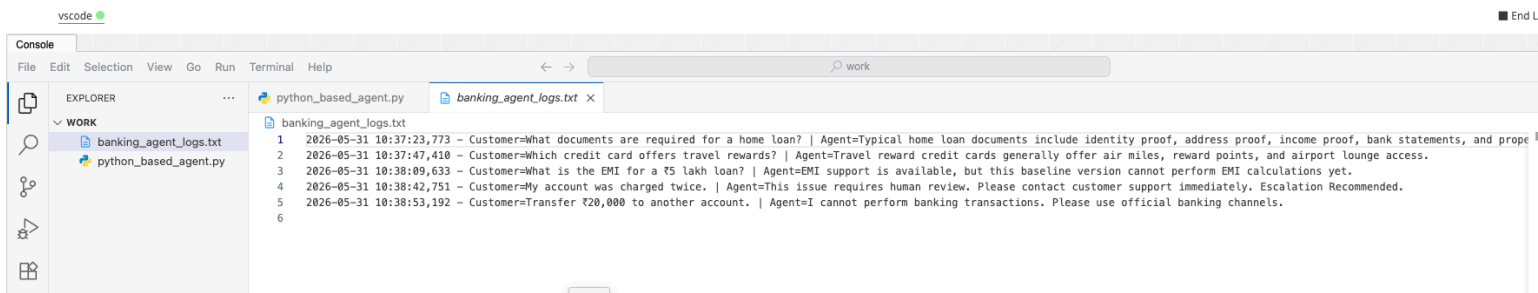
Customer: Transfer ₹20,000 to another account.
Agent: I cannot perform banking transactions. Please use official banking channels.

Customer: 

```

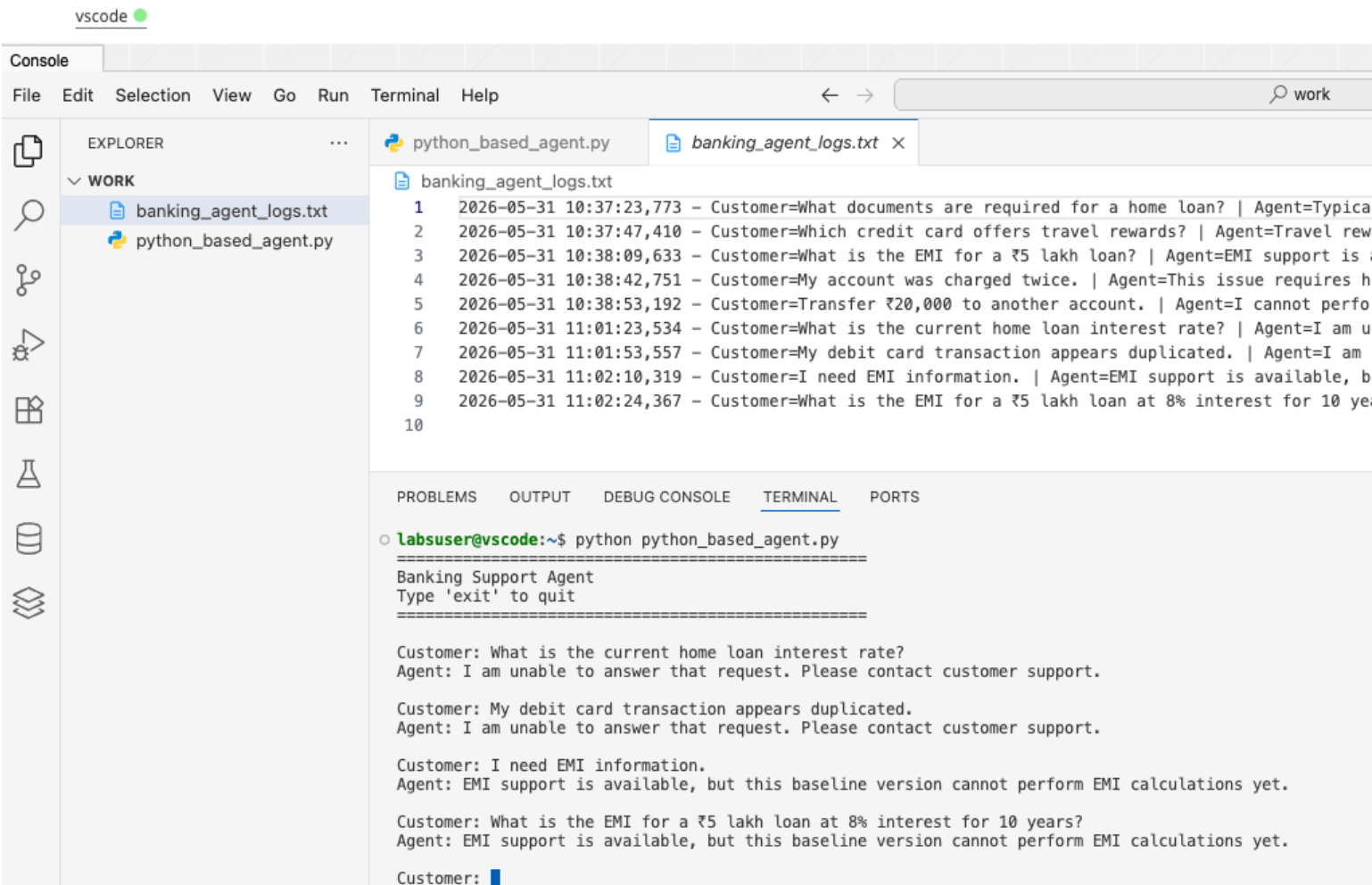
5. Sample Log output

The following snippet from the Vocareum shows the sample logout from the python rule based agent.



```
1 2026-05-31 10:37:23,773 - Customer=What documents are required for a home loan? | Agent=Typical home loan documents include identity proof, address proof, income proof, bank statements, and propo
2 2026-05-31 10:37:47,410 - Customer=Which credit card offers travel rewards? | Agent=Travel reward credit cards generally offer air miles, reward points, and airport lounge access.
3 2026-05-31 10:38:09,633 - Customer=What is the EMI for a ₹5 lakh loan? | Agent=EMI support is available, but this baseline version cannot perform EMI calculations yet.
4 2026-05-31 10:38:42,751 - Customer=My account was charged twice. | Agent=This issue requires human review. Please contact customer support immediately. Escalation Recommended.
5 2026-05-31 10:38:53,192 - Customer=Transfer ₹20,000 to another account. | Agent=I cannot perform banking transactions. Please use official banking channels.
6
```

6. Baseline Agent Limitations



```
1 2026-05-31 10:37:23,773 - Customer=What documents are required for a home loan? | Agent=Typical
2 2026-05-31 10:37:47,410 - Customer=Which credit card offers travel rewards? | Agent=Travel rew
3 2026-05-31 10:38:09,633 - Customer=What is the EMI for a ₹5 lakh loan? | Agent=EMI support is
4 2026-05-31 10:38:42,751 - Customer=My account was charged twice. | Agent=This issue requires h
5 2026-05-31 10:38:53,192 - Customer=Transfer ₹20,000 to another account. | Agent=I cannot perfo
6 2026-05-31 11:01:23,534 - Customer=What is the current home loan interest rate? | Agent=I am u
7 2026-05-31 11:01:53,557 - Customer=My debit card transaction appears duplicated. | Agent=I am
8 2026-05-31 11:02:10,319 - Customer=I need EMI information. | Agent=EMI support is available, b
9 2026-05-31 11:02:24,367 - Customer=What is the EMI for a ₹5 lakh loan at 8% interest for 10 ye
10
```

```
labuser@vscode:~$ python python_based_agent.py
=====
Banking Support Agent
Type 'exit' to quit
=====

Customer: What is the current home loan interest rate?
Agent: I am unable to answer that request. Please contact customer support.

Customer: My debit card transaction appears duplicated.
Agent: I am unable to answer that request. Please contact customer support.

Customer: I need EMI information.
Agent: EMI support is available, but this baseline version cannot perform EMI calculations yet.

Customer: What is the EMI for a ₹5 lakh loan at 8% interest for 10 years?
Agent: EMI support is available, but this baseline version cannot perform EMI calculations yet.

Customer:
```

Limitation 1: No Real Banking Knowledge Retrieval

Reason: The agent uses predefined rules and cannot retrieve information from banking policy documents.

Limitation 2: No Natural Language Understanding

Reason: The rule engine only recognises specific phrases such as "charged twice". Similar wording may not be understood.

Limitation 3: No Conversation Memory

Reason: The agent does not remember information from previous messages.

Limitation 4: No Calculations

Reason: The baseline version does not have calculator tools integrated.

7. Why This Version Is Insufficient for Real Users

Although the baseline agent demonstrates the basic workflow of a banking assistant, it is not suitable for production use.

The system has several limitations:

- Uses hard-coded rules instead of banking knowledge retrieval.
- Cannot understand many natural language variations.
- Cannot access policy documents.
- Cannot perform EMI calculations.
- Cannot maintain conversation history.
- Cannot verify information against banking records.
- Cannot adapt automatically to policy changes.
- Supports only a limited set of predefined questions.

As a result, customers may receive incomplete or unsupported responses for many real-world banking queries.

8. Future Improvements

Future phases will introduce more advanced AI capabilities:

1. Intent Classification
 - Automatically identify customer intent.
2. Retrieval-Augmented Generation (RAG)
 - Retrieve information from banking policy documents.
3. Tool Calling
 - Support EMI calculations and financial utilities.
4. Conversation Memory
 - Maintain context across multiple interactions.
5. Safety Guardrails
 - Improve handling of risky and sensitive requests.
6. Human Escalation Workflow

- Route fraud complaints and complex cases to support teams.
- 7. Evaluation Framework
 - Measure accuracy, hallucination rate, retrieval relevance, and latency.

9. Conclusion

A basic Banking AI Support & Advisory Agent was successfully implemented using Python and a rule-based approach. The agent accepts user input, generates banking-related responses, refuses unsafe requests, logs interactions, and demonstrates key limitations. This phase establishes the foundation for future enhancements involving retrieval, tool integration, memory, and advanced AI capabilities.